



ROBOCUPJUNIOR RESCUE MAZE 2026

TEAM DESCRIPTION PAPER

RC-UANL

Abstract

The robot was developed with the objective of overcoming the various challenges encountered while navigating a maze. To achieve this, multiple solutions were implemented in both the navigation and mechanical systems, allowing the robot to adapt to different situations within the environment.

The most important aspect of the system is the navigation, which incorporates multiple systems in the programming logic. This is because we incorporate ways to improve navigation, such as a constant alignment between the halls of the maze. This enables the robot to not lose more tiles between the navigation of the maze, also improving in some ways the turns of different degrees through the navigation, orientation and general orientation errors, reducing the accumulated errors between the navigation.

In addition to the software solutions, the robot includes several mechanical systems that enhance the navigation of the robot in the maze. These systems include a suspension mechanism as well as alignment mechanisms based on limit switches and the frontal TOF of the main board, which help maintain proper positioning throughout navigation.

During operation, the robot can identify various elements within its environment, such as victims and specific tile types, while also detecting and avoiding obstacles that may appear along its navigation, including stairs, bumpers and ramps being helped by custom tires made of silicon.

To integrate all these systems, a custom-designed PCB was developed. This board manages the connections between multiple sensors and electronic systems, including the OpenMV cameras, which are responsible for visual processing. Together with the various electronic and mechanical components, these systems provide the robot with the ability to respond to changes in its surroundings, enabling reliable operation across different maze difficulties.

1. Introduction

a. Team

- **Stefany Giselle Colunga Rangel:** She is the captain of the team responsible for programming, focusing primarily on robot movement algorithms and the development of software improving in the navigation path. She has extensive experience and has participated in multiple OnStage tournaments since 2019. Having the opportunity to represent Mexico in the onstage category at RoboCup 2023 in Bordeaux also being its first time competing in the Maze category.
- **Diego Emilio Evangelista Gómez:** He is responsible for the robot's mechanical systems and solutions of the navigation and victim detection mechanisms, developing and integrating the physical systems within the robot. He has been participating in tournament robotics since 2022, competing in the Rescue Line and OnStage categories, with this being his first participation in Rescue Maze. Having the opportunity to represent Mexico in the onstage category at RoboCup 2025 in Salvador, Bahia.
- **Axel Uriel Hernández Labrada:** He is responsible of the programming focusing on the victim detection systems and in some tasks from the navigation and detection of the maze, having experience in multiple tournaments since 2023 in both soccer categories, rescue line and rescue maze being its first time representing Mexico in international tournaments.
- **Rogelio Andrade López:** He is responsible of the electronics systems working in the design of the main board that incorporates all the sensors having experience in multiple rescue line category in national tournaments since the 2023 being its first tournament in rescue maze and time representing Mexico.

2. Project Planning

a. Requirements definition

The robot requirements were established according to the challenges and constraints imposed by the Rescue Maze competition. The system was designed to navigate efficiently through the maze while maintaining reliability and adaptability under different operating conditions. The mechanical design was intended to provide easy maneuverability, robustness, and accessibility for maintenance, allowing repairs and modifications to be performed quickly when necessary. This design philosophy also applies to electronic architecture, which was planned as a flexible and modular system capable of accommodating future improvements and component replacements. Similarly, the software architecture was developed using independent modules, ensuring that issues or modifications in one subsystem could be addressed without negatively affecting the operation of the remaining programming packages

b. Overall Project Plan

Our primary objective as a team is the successful development of a robotic system capable of completing the various challenges, tasks, and requirements established by RoboCup. We aim to identify and implement the most effective solutions to the problems encountered throughout the development process, ultimately resulting in a robot that operates efficiently, reliably, and autonomously. In addition to our technical goals, we seek to promote collaboration and integration among all team members while simultaneously developing new skills and acquiring valuable knowledge through the project.

The requirements established for the robot include a compact design that facilitates navigation while maintaining an appropriate center of gravity and balanced weight distribution. The robot is primarily manufactured using 3D-printed components, allowing efficient implementation of the CAD design while maintaining low manufacturing costs. Careful consideration is given to the selection and placement of sensors and the vision system to ensure the successful execution of the robot's tasks. Furthermore, custom silicone wheels are designed to improve traction and overall navigation performance, and a custom main PCB is developed to integrate the various electronic subsystems into a single platform.

From the software perspective, the system is developed using C++ and MicroPython, incorporating multiple communication protocols to enable interaction between the different hardware and software modules.

Collectively, these systems must be capable of autonomously exploring the maze, identifying important environmental features such as colored tiles and victims, generating a map of the explored area, and successfully returning to the starting point while complying with the competition rules.

The entire development process is planned to be completed within a six-month period. During this time, tasks are organized into weekly milestones to ensure structured progress and effective project management. The project is carried out while considering budget limitations and component availability, with a strong focus on optimizing resources and development time. To support this process, a detailed project schedule has been created, which can be seen in *Figure 1*.



Figure 1. Project planner through the weeks

As a team, we defined and agreed upon a structured set of development phases aligned with our project objectives. This organization was essential for the development; it allowed us to begin with a research stage followed by the creation of a structural prototype of the robot. Based on this initial design, we were able to adapt the electronics and ensure that software development progressed in parallel with the chosen hardware architecture.

Throughout the process, multiple tests were conducted to evaluate the robot's functionality and performance following the previously established workflow. These evaluations directly influenced the final selection of components, including possible changes in sensors, software strategies, or mechanical designs. The main objective of these iterations was to continuously improve the performance of the developed robot.

b. Integration Plan

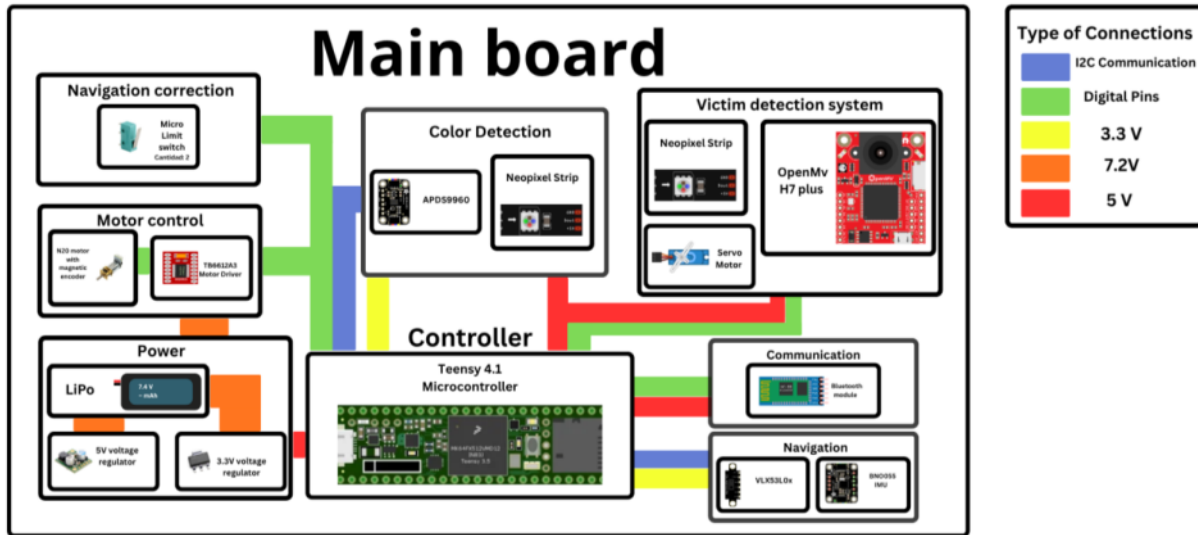


Figure 2. final architecture of the electronics

To address all the challenges presented in the competition, solutions were progressively developed as new problems arose throughout the project. Beginning with the hardware, several iterations of the robot were designed to properly allocate the space required for the integration of all subsystems and components.

Throughout the development process, several challenges were encountered in both electronics and software. These included failures in the I²C communication network, inadequate power distribution, and issues related to the structure and organization of the software architecture. Each of these problems was systematically analyzed and improved through multiple design revisions and testing cycles.

As a result of this iterative development process, the system evolved into the final architecture shown in **Figure 2**. In this configuration, each component successfully fulfills its intended function while maintaining reliable communication and coordination with the rest of the system, enabling efficient and stable robot operation.

3. Hardware

a. Mechanical Design and Manufacturing

Main Structure

The mechanical design of the robot was developed having the purpose of minimizing navigation difficulties within the maze and improving tasks of detection. This was achieved through an efficient distribution of components as well as the implementation and development of several mechanical subsystems focused on maximizing efficiency and reliability. The entire mechanical system was designed in SolidWorks, utilizing a variety of materials and manufacturing processes.

Actuators and Power Train

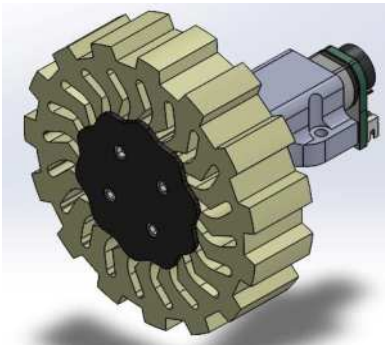


Figure 3. actuator and wheel

Starting with the robot's actuators, the platform uses micro gear motors due to their high torque-to-size ratio and relatively low power requirements. These characteristics make them well suited for a compact mobile robot operating in a complex environment. For the rescue kit deployment system, servo motors were selected for this task which provides greater torque and positioning accuracy than while maintaining low energy consumption. These characteristics were essential for the reliable deployment of rescue kits during victim detection tasks.

Closely related to the actuator system is the wheel design. The wheels were specifically developed to conform to varying terrain conditions using and surface features that improve traction. This design allows the robot to better navigate flat surfaces, ramps, stairs, and other obstacles commonly found within the competition environment.

The wheels were manufactured using 3D-printed molds and subsequently cast in silicone, providing both flexibility and grip. In addition having a PLA disc to provide deformations in the wheels that not should be there adding different types of texture in the way of how the tyres interact with the environment, a digital model of the wheels was developed to perform simulations and evaluate their performance prior to fabrication, allowing for a more detailed analysis of their behavior and functionality, as shown in *Figure 3*.

Subassemblies – modules:

Suspension Mechanism

One of the robot's key mechanical subsystems is the suspension mechanism. This system incorporates smooth axle and shaft collars which, together with a custom chasis, create a rotational mechanism capable of adapting to changes in the terrain like bumpers or ramps and stairs. The primary purpose of this system is to absorb the impact of different terrains and maintain stable contact with the ground during navigation this mechanism can be seen in *Figure 4*.

Rescue Kit Deployment System

Regarding victim assistance, the rescue kit deployment system was designed to be as compact as possible to fit within the robot's dimensions and be easy to calibrate and repair. The mechanism is based on a rack-and-pinion system, where a servomotor drives a pinion gear that pushes a rack. The rack then transfers the motion to the rescue kits stored inside the deployment towers which, when it moves, allows the kit to fall and be released when it's required, as illustrated in *Figure 5*.

Correction System

Another important subsystem is the correction system, which uses two limit switches to assist the robot in detecting and avoiding larger obstacles, such as bricks. Additionally, the system helps the robot realign itself whenever accumulated errors from the IMU affect its positioning. The bumper is manufactured from PETG, providing the rigidity required to withstand impacts and reliably activate the limit switches regardless of the point of contact. The design of this mechanism can be seen in *Figure 6*.

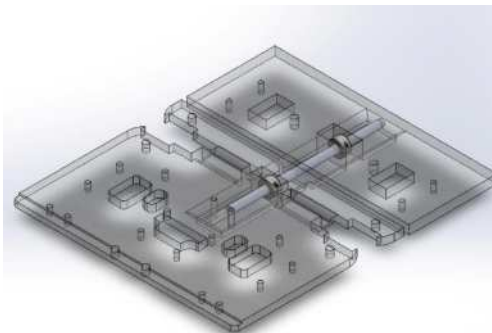


Figure 4. Suspension mechanism

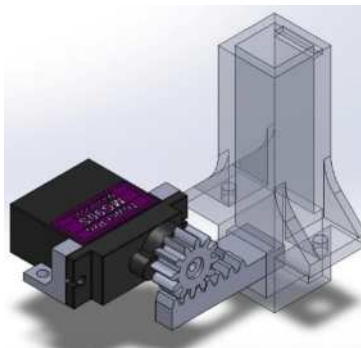


Figure 5. Servo mechanism

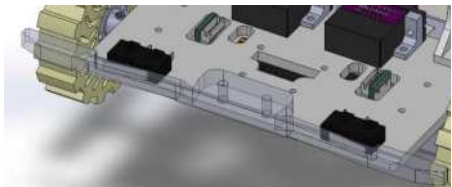


Figure 6. Correction System

b. Electronic Design and Manufacturing

To support the robot's electronic architecture, a custom Printed Circuit Board (PCB) was developed. The primary objective of this design was to optimize the available space inside the robot while providing an organized and reliable method for connecting all integrated components. The PCB serves as the central hub for communication, power distribution, and sensor integration. Its implementation, design details, and main components are shown in **Figure 7**.

Sensors Used

The primary sensors used for navigation are the VL53L0X sensors, which are Time-of-Flight (ToF) distance sensors capable of providing highly accurate distance measurements. The robot incorporates nine VL53L0X sensors distributed around the four sides of the PCB, allowing comprehensive environmental perception. Since these sensors share the same I²C address by default, their XSHUT pins are used to individually initialize and manage communication with each device.

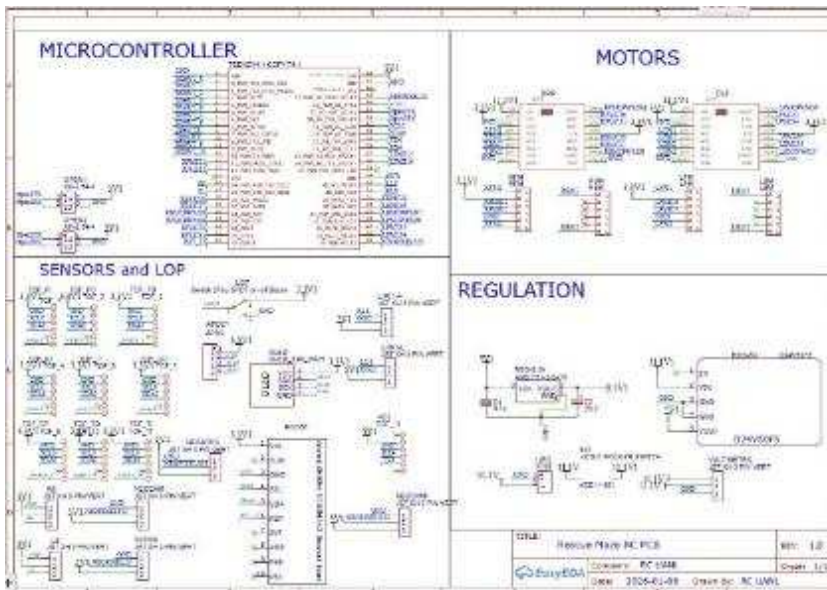


Figure 7. Schematic diagram of the PCB

competition-specific floor features. All these sensors communicate through the I²C protocol, allowing efficient integration within the system.

For victim detection, the robot uses an OpenMV H7 Plus camera module. The camera communicates with the Teensy 4.1 through UART communication protocol this is because it's an easy way to communicate the two systems and will permit us to have more possibilities on the management of the deployment of the kits through the navigation of the maze.

Main Controller

The robot's main controller is a Teensy 4.1, which manages all sensor inputs, actuator outputs, and communication interfaces. This microcontroller was selected due to its large number of digital and analog pins, high processing performance, and support for multiple communication protocols.

One of the key factors in selecting the Teensy 4.1 was its ability to handle large amounts of data arriving simultaneously from multiple sensors without experiencing performance issues or system lockups. This capability is particularly important for coordinating navigation, victim detection, mapping, and actuator control in real time. Furthermore, its flexible pin functionality allows efficient implementation of encoder interfaces, which play a critical role in the robot's navigation system.

Power Subsystem

To complement the distance sensors, the robot also uses limit switches connected to digital input pins. These switches assist with navigation correction and physical alignment when the robot interacts with walls or other structures within the maze.

For orientation and motion sensing, a BNO055 inertial measurement unit (IMU) is employed as the primary navigation sensor. This device provides accurate heading information for turns and is also used to detect ramps and stairs encountered during maze exploration.

Additionally, an APDS9960 sensor is used to detect special floor tiles such as checkpoints, danger zones, and other colored markers. Multiple calibration procedures were performed to improve the identification of silver tiles and other

The robot is powered by a 2-cell 2200 mAh LiPo battery, which serves as the primary energy source for the entire system. Power is distributed to the various subsystems through dedicated 5 V and 3.3 V voltage regulators, ensuring stable operation of the electronics.

The drive motors are powered directly from the battery to maximize available torque and overall drivetrain performance, while the regulated power rails supply the sensors, controller, camera, and other low-voltage electronics.

Actuators

The robot uses N20 micro gear motors, selected due to their favorable balance between electrical characteristics, torque output, and compact size. These features make them well suited for a maze-solving robot where both performance and space efficiency are critical. The motor specifications are summarized in **Figure 8**.

For the rescue kit deployment mechanism, the robot utilizes a pair of SG90 servo motors, which provide the positioning accuracy and simplicity required for reliable kit deployment during victim assistance tasks.

Nominal Voltage	No-Load Speed	Gear Ratio	Peak Torque	Peak Current
6VDC	100 RPM	150:1	2.75 Kg-cm	0.55 A

Figure 8. Specifications of the micromotors

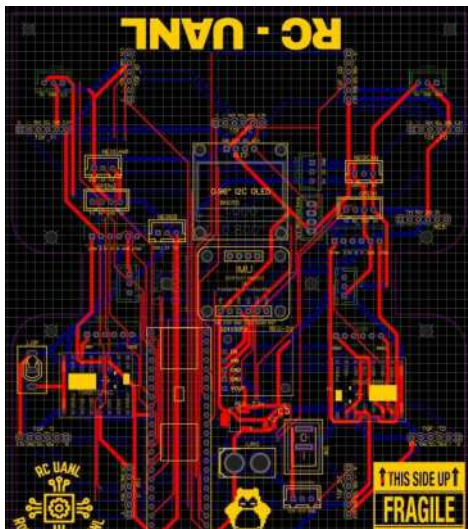


Figure 9. Layout of the PCB

PCB Design and Integration

All these subsystems were integrated into a custom-designed PCB developed to efficiently combine the various electronic systems used throughout the project. The board centralizes the navigation, power distribution, sensor, actuator, and indicator systems, simplifying both hardware organization and communication between the different modules of the robot. For physical implementation, a dedicated PCB layout was created, which can be seen in Figure 9.

The PCB was designed using EasyEDA and was specifically sized to fit within the robot's chassis, allowing the available space to be optimized while maintaining an organized component arrangement. One of the most significant improvements was the direct integration of several sensors, including the laser distance sensors and color sensors, which were soldered directly onto the PCB. This approach improved connection reliability while reducing wiring complexity.

To facilitate assembly, maintenance, and future modifications, JST connectors and pin headers were incorporated throughout the design.

These connectors allow modules to be connected, removed, or replaced quickly and efficiently without requiring extensive rewiring.

Additionally, the PCB includes dedicated switches for both the robot's main power and the Lack of Progress function required by the competition rules. The system also incorporates NeoPixel LEDs used to illuminate the floor and surrounding walls, improving environmental detection and sensor performance. Furthermore, several status LEDs provide visual feedback regarding system operation, while an SSD1306 OLED display presents real-time information about the robot's actions, sensor readings, and victim detections, allowing operators to easily monitor the robot's behavior during testing and competition.

4. Software

a. General software architecture

The robot's software is primarily developed using two development environments. The first is Arduino IDE, which serves as the main platform for implementing the robot's control system. This environment is responsible for navigation, danger zone and checkpoint detection, actuator control, sensor management, environmental perception, and the processing of victim detection interruptions. As a result, it functions as the robot's primary control program.

The second development environment is OpenMV IDE, which is dedicated to victim and target detection. The OpenMV camera communicates with the main PCB through UART, using the OpenCV's UART 1 pins to notify when a victim is detected and sending to the Teensy a number according to the interpretation of the victim.

Programming Logic Flow

The software execution begins with the calibration of the APDS9960 sensor for tile detection and the initialization of the BNO055 IMU according to the robot's starting orientation. Once initialization is complete, the navigation algorithm begins analyzing the environment using the distance measurements provided by the ToF sensors.

The navigation strategy is based on Depth-First Search (DFS) with absolute orientation. Using the front-left and front-right sensors, the robot determines which paths remain available for exploration. As the robot moves through the maze, the algorithm records every explored node and encountered event.

In addition to DFS, the robot also utilizes the Dijkstra algorithm to calculate efficient routes when required, particularly when returning to the starting position. Navigation decisions are processed by Teensy 4.1, which controls the motors and commands the robot to advance one tile at a time, corresponding to approximately 30 cm. The distance traveled is calculated using the encoder ticks recorded by each motor.

During movement, several interrupt-driven events may occur. These include victim detections reported by the camera system, tile detections reported by the color sensor, and obstacle detections generated by the limit switches. After processing each event, the robot resumes its navigation task

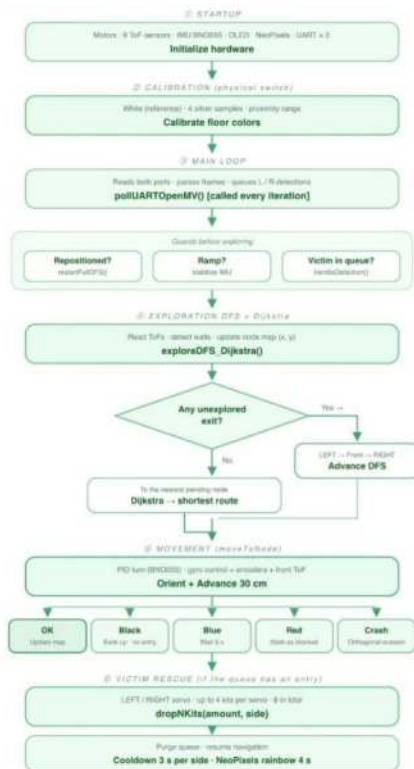


Figure 10. Programming flowchart

To improve positioning accuracy, the robot periodically performs physical realignment procedures. After detecting three surrounding walls, the robot intentionally uses its bumper system to align itself against a wall, correcting accumulated errors caused by wheel slippage, 30 cm movement inaccuracies, or IMU drift during turns. This process is repeated throughout the exploration of the maze.

All actions performed by the robot are displayed on the onboard OLED display, allowing real-time monitoring of the robot's state and decision-making process. An overview of the software architecture can be observed in **Figure 9**.

Navigation Strategy

To maximize exploration coverage and achieve the highest possible score, the robot combines Depth-First Search (DFS) and Dijkstra's Algorithm. DFS is used to explore as much of the maze as possible, while Dijkstra's Algorithm is employed to determine the most efficient route back to the starting position once exploration has been completed.

The navigation system maintains a matrix representation of the maze that is continuously updated as new tiles are explored. Every discovered tile is stored within this map structure, enabling the robot to distinguish between explored and unexplored regions.

Using graph theory principles, each tile is treated as a node within a graph. The DFS algorithm systematically explores each branch as deeply as possible before backtracking, ensuring comprehensive coverage of the environment while maintaining awareness of unexplored paths using absolute orientation.

APDS9960 Calibration

For color detection, a calibration process was performed based on the comparison of measured values, color differences, and RGB-C normalization techniques. These procedures allow the robot to reliably distinguish between the various tile types used in the competition, including checkpoints, danger zones, and silver tiles.

ToF, IMU, and PID Accuracy Improvements

To address the inherent inaccuracies of the navigation sensors, several calibration and correction procedures were implemented.

For the VL53L0X ToF sensors, raw distance measurements are obtained in millimeters. Calibration factors were then applied to compensate for measurement deviations by increasing or decreasing the reported values according to experimentally measured distances. This process significantly improved distance estimation accuracy during wall following and navigation.

BNO055 IMU Implementation

Robot orientation is obtained using the `getEvent ()` function provided by the BNO055 library. During normal navigation on flat surfaces, the robot primarily uses the Euler angle corresponding to the X-axis to determine heading. When traversing ramps or stairs, the Z-axis angle is used instead, as it provides more reliable information regarding the robot's inclination.

The measured angles are processed directly in degrees and passed through a normalization function, `normalize360()`, which constrains all values to the 0° – 360° range. This ensures consistency throughout the navigation calculations.

To calculate the angular error between the robot's current heading and the desired heading, the difference between the two angles is computed. The resulting value is then adjusted to remain within the range of -180° to 180° , ensuring that the robot always rotates through the shortest possible path and avoids unnecessary turns.

PID Controller Implementation

The PID controller serves as the primary method for regulating the robot's navigation variables and maintaining stable motion throughout operation. Its main applications are found in the lateral alignment system and in the processing of IMU data, where it enables more accurate corrections that help keep the robot centered within the maze corridors.

In addition, the PID controller determines both the response intensity and correction speed used when rotating toward target angles during navigation. This allows the robot to perform smoother, more precise, and controlled turns while minimizing overshoot and oscillations.

The controller is also utilized during the traversal of ramps and stairs, where it contributes to maintaining the robot's stability and orientation despite changes in inclination. By continuously adjusting the control output according to sensor feedback, the PID system improves the robot's ability to navigate uneven terrain while preserving accurate positioning and heading throughout the exploration process.

Victim management system:

The software developed for victim detection begins by acquiring images from the OpenMV camera. The camera first decides using a state-based detection system that operates in two modes: letter victims and cognitive victimis mode.

In letter detection mode, a convolutional neural network based on the MobileNet architecture is used. The captured image is processed by the network, which performs inference and communicates its decision to the Teensy microcontroller through UART pins indicating whether a victim has been detected and communicates the number of rescue kits that must be deployed, an example of this can be seen in *Figure 10*.

The cognitive Targets are based on a geometry-based Computer Vision algorithm processes targets without heavy neural networks through three strict stages, an example of a cognitive target is shown in *Figure 11*:

- **Detection & Proximity Validation:** Uses color blob detection and geometric filtering to identify perspective-distorted ellipses. To comply with the 15 cm deployment rule, analysis only triggers if the ellipse's minor axis exceeds a safe pixel threshold.
- **LAB Space Classification:** Samples pixels across 5 concentric rings. By calculating the squared Euclidean distance to pre-calibrated LAB centroids, it classifies colors robustly against varying venue lighting: Blue (2), Green (1), Yellow (0), Red (-1), or Black (-2).
- **Health Status Evaluation:** Sums the 5 ring values to determine the exact kit requirement (Harmed = 2, Stable = 1, Unharmed = 0). Invalid mathematical sums are instantly rejected as false positives.



Figure 11. Letter victim detection



Figure 12. Cognitive victim detection

b. Innovative solutions

Obstacle Avoidance System

Among the developed functions is an obstacle avoidance system designed to handle larger objects, such as bricks, using limit switches combined with a front bumper mechanism. This system enables the robot to detect collisions with objects that are not part of the standard maze layout, preventing navigation errors and unintended movements.

When a collision is detected, the robot immediately stops and performs a corrective maneuver. This preserves navigation accuracy and allows the robot to safely reorient itself before continuing its path through the maze.

Error Accumulation Correction System

Another important function is the robot's alignment system when in a static state. This system uses the BNO055 IMU together with the ToF sensors (VL53L0X) to correct errors caused by environmental variations and small deviations accumulated during movement. The correction process continuously reads these sensors to maintain an appropriate distance from surrounding walls while minimizing angular error.

This realignment improves navigation performance by allowing the camera to maintain a balanced field of view on both sides of the tile. In addition, the robot performs periodic position corrections after traversing a defined number of tiles using the limit switches, helping to reduce drift and ensure consistent positioning throughout maze exploration.

30 cm Tile Displacement Measurement

An additional innovation is the use of magnetic encoders to accurately verify the robot's 30 cm displacement per tile, improving movement precision and overall navigation reliability.

Cognitive Target Detection System

One of the most innovative solutions implemented is the Cognitive Target Detection System, designed to identify and classify cognitive victims without relying on computationally expensive neural networks. Instead, it uses a lightweight computer vision approach that enables fast and efficient detection.

The process begins with color blob detection and geometric filtering to identify perspective-distorted ellipses corresponding to cognitive targets. To comply with the 15 cm rescue kit deployment rule, analysis is only performed when the detected ellipse exceeds a predefined minor-axis pixel threshold, ensuring proximity validity.

Once a valid target is detected, classification is performed in the LAB color space. Pixel samples are taken across five concentric rings inside the target, and the squared Euclidean distance is calculated against pre-calibrated LAB color centroids. This provides robust recognition under varying lighting conditions, enabling reliable distinction between blue, green, yellow, red, and black segments.

Finally, the values associated with each detected ring are combined to evaluate the victim's health status and determine the required rescue kit response. The algorithm validates the resulting combination and rejects invalid configurations as false positives. This multi-stage geometric and color-based approach provides a computationally

efficient alternative to neural networks, enabling rapid decision-making while maintaining high detection accuracy during maze exploration.

5. Performance evaluation

Testing and Performance Evaluation

After completing the robot's development, its performance was evaluated on a maze competition track. The robot was tested on multiple maze configurations with progressively increasing difficulty in order to determine whether further improvements were required. In addition, each individual subsystem was tested separately before being fully integrated into the complete system to verify its effectiveness.

Camera tests were also conducted with the robot operating on the track, along with evaluations of the required illumination conditions. Serial communication outputs were monitored to analyze the data transmitted by the robot, allowing for the identification of specific areas that needed improvement to optimize overall performance.

One of the main issues identified was the camera positioning, as its placement and angular orientation were not optimal to cover the full detection range required for reliable victim identification. Another challenge was the detection of silver tiles, which was initially confused with white tiles during operation on the track. This issue was resolved through multiple recalibrations of the APDS sensor. Finally, adjustments were made to the camera focus to prevent misclassification of distant objects as valid detections.

After several iterations of testing and refinement, the robot demonstrated effective performance across all proposed test scenarios as we can see on *figure 12 and figure 13*.

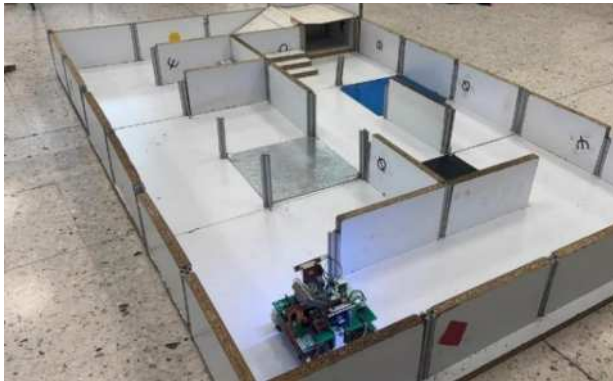


Figure 13. Robot in maze



Figure 14. Robot in maze

6. Conclusion

The planning, integration of components, and development of well-defined strategies, together with the implementation of multiple subsystems within the robot, enabled a successful development process in which various modules worked together efficiently. This proper integration was key to achieving a compact system capable of executing different functions in a coordinated manner during navigation.

Throughout the project, we not only met the established objectives but also strengthened our technical skills and acquired new knowledge in areas such as programming, electronics, and mechanical design. This process represented a continuous challenge that promoted constant improvement within the team, both individually and collectively.

References

- Adafruit Industries. (2024). *Adafruit APDS9960 breakout: Overview*. Adafruit Learning System. <https://learn.adafruit.com/adafruit-apds9960-breakout/overview>
- Adafruit Industries. (2025). *Adafruit BNO055 absolute orientation sensor: Arduino code*. Adafruit Learning System. <https://learn.adafruit.com/adafruit-bno055-absolute-orientation-sensor/arduino-code>
- GeeksforGeeks. (s.f.). *Depth first search or DFS for a graph*. <https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>
- OCI Labs. (s.f.). *Taller avanzado | Algoritmo Dijkstra - implementación* [Video]. YouTube. <https://youtu.be/vs14IxuW33I>